

DOCKER

Cheat Sheet — Key Concepts for Interviews & Daily Use

Docker solves the “works on my machine” problem by packaging an application together with its dependencies, libraries, and configuration into a single portable unit called a container.

★ Core Concepts

- Image = blueprint (read-only template); Container = running instance of an image
- Containers share the host OS kernel → much lighter than virtual machines
- Dockerfile → set of instructions used to build an image

```
docker build -t myapp:1.0 .  
docker run -d --name myapp_container myapp:1.0
```

- Multiple containers can run from the same image, independently

Image vs Container

- Image: static, versioned, stored in a registry (e.g. Docker Hub, ECR)
- Container: ephemeral, adds its own writable layer on top of the image
- Stopping a container does not delete it — it can be restarted
- Removing a container destroys its writable layer and any unsaved data

Image Layers

- Each instruction in a Dockerfile creates a new image layer
- Layers are cached — unchanged layers are reused on rebuild
- Layers are read-only; the container adds one writable layer on top (copy-on-write)

✓ **Tip:** Order your Dockerfile instructions from least-frequently-changed to most-frequently-changed to maximize cache hits.

✂ Dockerfile Instructions

Instruction	Purpose
FROM	Specifies the base image to build on
RUN	Executes a command at build time (e.g. install packages)
COPY / ADD	Copies files into the image. ADD can also extract archives and fetch URLs
WORKDIR	Sets the working directory for subsequent instructions
ENV	Sets environment variables available inside the container
EXPOSE	Documents which port the container listens on (does not publish it)
CMD	Default command/args for the container; overridable at runtime
ENTRYPOINT	Fixed executable that always runs; CMD becomes its default arguments

USER	Sets the user the container runs as (avoid running as root)
------	---

⚠ Build Best Practices

- Use a `.dockerignore` file to exclude unnecessary files from the build context
- Use multi-stage builds → build dependencies are not shipped in the final image
- Prefer slim/alpine base images → smaller size, smaller attack surface
- Pin specific versions for base images and packages for reproducibility

🌐 Networking

- Default network: bridge → containers on the same bridge can talk via IP
- Custom network (docker network create) enables DNS resolution by container name
- Port mapping: `-p host_port:container_port` publishes a container port to the host
- host network → container shares the host's network stack directly
- none network → container has no networking at all

💾 Volumes & Storage

- Containers are ephemeral — data is lost when removed unless persisted
 - Volume: managed by Docker, stored outside the container's writable layer
 - Bind mount: maps a host directory directly into the container
- ```
docker run -v myvolume:/data ...
docker run -v /host/path:/container/path ...
```
- Volumes survive container deletion; bind mounts reflect host file changes live

✓ **Tip:** Use named volumes for databases and persistent app data; use bind mounts for local development workflows.

## 🔄 Container Lifecycle Commands

```
docker ps # running containers
docker ps -a # all containers (incl. stopped)
docker stop <name> # graceful stop (SIGTERM, then SIGKILL)
docker rm <name> # remove a stopped container
docker exec -it <name> bash # shell into a running container
docker logs -f <name> # stream container logs
```

## 📋 Docker Compose

- YAML file (`docker-compose.yml`) defines multi-container applications
- ```
docker compose up -d # start all services in the background
docker compose down  # stop and remove containers, networks
```
- Services communicate using their service name (built-in DNS)
 - `depends_on` controls startup order — NOT readiness

⚠ *`depends_on` only waits for a container to start, not for the app inside it to be ready. Use healthchecks to wait for actual readiness.*

Security & Resource Limits

- Avoid running containers as root — set a non-root USER in the Dockerfile
- --memory and --cpus flags limit how much of the host's resources a container can use
- Scan images for known vulnerabilities (e.g. docker scan, Trivy)
- Avoid storing secrets in images — use environment variables, secrets managers, or Docker secrets

Image Management

```
docker images          # list local images
docker tag src:tag dest:tag # create alias/version tag
docker push / docker pull # registry operations
docker system prune -a   # clean up unused images, containers, networks
```

Advantages of Docker

- Consistency across dev, staging, and production environments
- Fast startup compared to virtual machines
- Easy horizontal scaling, especially with orchestrators like Kubernetes or Swarm
- Simplifies dependency management and onboarding for new developers